PROJECT REPORT

Ishan Choudhary 25BCE10753

1. Introduction

Financial management is one of the significant aspects in our day-to-day life. However, people face difficulties in keeping track of their daily income and expenses. Keeping track of expenses in traditional ways is very time-consuming and difficult

In order to overcome these difficulties in financial management, the Smart Expense Tracker project was created by me. This project is a simple yet powerful command-line-based system that allows users to record their financial data. This system helps users to gain more insight about their expenses and manage them more effectively

2. Problem Statement

In the present busy world, people cannot keep track of their expenses. This leads to poor financial planning and unnecessary expenses. There is a need to develop a system that can:

- Efficiently record transactions
- Categorize the expenses
- Show a summary of the income and expenses
- Show the data in a visual format so that users can easily understand the data

3. Objectives

The objectives of this project are:

- To design a system to record the income and expenses
- To categorize the expenses so that users can easily maintain their expenses
- To calculate the total income, expenses, and the difference
- To show category-wise expenses
- To show expenses in a visual format using graphical representation ● To store the data persistently in the database

4. Scope of the Project

The Smart Expense Tracker is designed for individual users who want to manage their daily finances efficiently. The application can be extended in the future with advanced features such as:

- Graphical User Interface (GUI)
- Budget alerts and notifications
- Data export to Excel/PDF
- Mobile or web-based integration

5. Technologies Used

- **Programming Language:** Python ● **Database:** SQLite ●
  Libraries Used:
  - Pandas (for data analysis)
  - Matplotlib (for data visualization)
- **Development Environment:** Command Line Interface (CLI)

6. System Design

The project follows a modular approach, where different functionalities are divided into separate modules:

Modules Description:

● **Main Module (main.py)**

Controls the flow of the application and displays menu options. ●

Add Transaction Module (add_transaction.py)

Handles user input and stores transaction data in the database. ●

View Summary Module (view_summary.py)

Fetches data and displays total income, expenses, and category-wise summary. ●

Visualization Module (visualization.py)

Generates bar charts to represent expenses visually. ●

Utility Module (utils.py)

Handles database connection and initialization.

7. System Architecture

User Input → Processing (Python Modules) → Database (SQLite) → Output (Summary/Graph)

8. Implementation Details

1. The application starts by initializing the SQLite database 2.
The user is presented with a menu to choose actions
3. When adding a transaction:
    - User inputs type, category, amount, and date
    - Data is stored in the database
4. When viewing summary:
    - Data is retrieved using SQL queries
    - Processed using Pandas
5. When visualizing the data:
    - Expense data is grouped by category
    - A bar chart is generated using Matplotlib

9. Features of the System

- Easy-to-use command line interface
- Add income and expense transactions
- Categorize expenses
- View financial summaries
- Visualize expenses using graphs
- Persistent data storage using SQLite

10. Advantages

- Simple and lightweight
- No internet connection required
- Easy to understand and use
- Helps in financial planning
- Modular and extendable

11. Limitations

- No graphical user interface (CLI-based)
- No authentication or user login system
- Limited to single-user usage
- No automatic data backup

12. Challenges Faced

- Handling invalid user inputs
- Managing database connections properly
- Displaying data clearly in the terminal
- Generating accurate visualizations

13. Future Enhancements

The following features can be added in future versions:

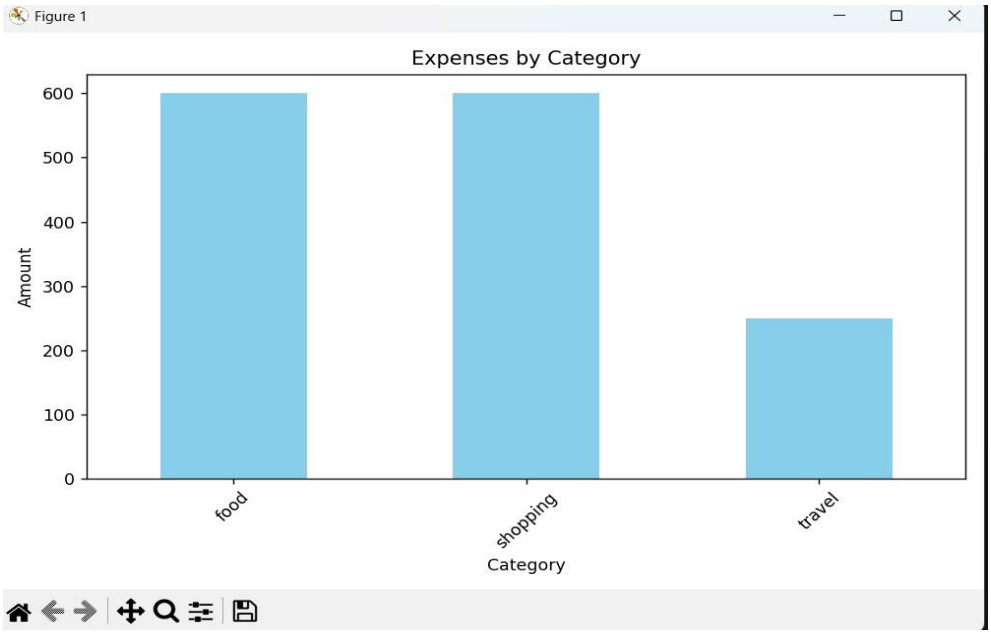
- GUI using Tkinter or PyQt
- Mobile or web-based application
- Budget tracking and alerts
- Export reports to PDF/Excel
- Multi-user support with login system

14. Screenshots

Below are the screenshots of the entry of expenses, income and the graph which were the outputs of the Smart Expense Tracker application

```
===== Smart Expense Tracker =====
1. Add Transaction
2. Summary
3. Visualize
4. Exit
Enter ur choice: 1
Enter type (income/expense): expense
Enter category: shopping
Enter amount: 600
Enter date (YYYY-MM-DD format): 2026-03-14
Transaction added

===== Smart Expense Tracker =====
1. Add Transaction
2. Summary
3. Visualize
4. Exit
Enter ur choice: 2

===== SUMMARY =====
Total Income   : 40000.0
Total Expense  : 1450.0
Balance        : 38550.0

===== CATEGORY-WISE =====
type     category
expense  food          600.0
         shopping      600.0
         travel        250.0
income   salary      40000.0
```

```
===== Smart Expense Tracker =====
1. Add Transaction
2. Summary
3. Visualize
4. Exit
Enter ur choice: 1
Enter type (income/expense): expense
Enter category: food
Enter amount: 400
Enter date (YYYY-MM-DD format): 2026-03-14
Transaction added

===== Smart Expense Tracker =====
1. Add Transaction
2. Summary
3. Visualize
4. Exit
Enter ur choice: 1
Enter type (income/expense): income
Enter category: salary
Enter amount: 20000
Enter date (YYYY-MM-DD format): 2026-03-01
Transaction added

===== Smart Expense Tracker =====
1. Add Transaction
2. Summary
3. Visualize
4. Exit
Enter ur choice: 1
Enter type (income/expense): expense
Enter category: travel
Enter amount: 250
Enter date (YYYY-MM-DD format): 2026-03-14
Transaction added
```



15. Conclusion

The Smart Expense Tracker effectively fulfills the purpose of helping users manage their personal finances in an efficient manner. The project shows the practical application of Python programming, database management, and data visualization techniques.
The project is successful in terms of functionality, and the code is clean and easy to read. The database is properly implemented, and the project is well-organized. The project is a good example of Python programming, database management, and data visualization techniques.

16. References

1. Python Official Documentation
2. Pandas Documentation
3. Matplotlib Documentation
4. SQLite Documentation